

Teknik Mimari ve Stack Seçimi

Mobil katman

Framework: React Native (Expo), seçimin ana nedeni proje isterlerinde React Native kullanılmasının istenmesidir. Cross-platform desteği ve bizim de sahadan fotoğraf almak için kullanacağımız JavaScript kütüphanelerinden (npm) gelen geniş ekosistemi en büyük avantajlarıdır. Ayrıca Expo altyapısı sayesinde Xcode ve Android Studio gibi araçlara olan bağımlılığın azalması geliştirme sürecini hızlandırmaktadır. Sahadan fotoğraf alma gibi cihaz özelliklerine resmi Expo paketleri üzerinden daha stabil şekilde erişilebilmesi de tercih edilme sebeplerinden biridir.

State management: Zustand, görev durumunu ve kullanıcı oturumunu yönetmek için seçilmiştir. uygulamamız MVP ölçeğinde olduğundan karmaşık dosya/şablon kod yükünden kaçınılmıştır. Ayrıca sadece durumu değişen bileşenleri tetikleme özelliği sayesinde gereksiz re-render sorunlarının önüne geçilmiştir.

Kütüphaneler:

expo-image-picker, kanıt fotoğrafını doğrudan cihaz kamerasını açarak alabilmek için seçilmiştir. Böylece özel kamera ekranı sunan kütüphanelerin karmaşık izin ve kodlama süreçlerini basitleştirmiş oluruz. Ayrıca fotoğraf çekim anında kalite ve boyut ayarlamasına da imkân tanır.

axios, promise tabanlı HTTP istemcisidir. Backend API'ya giden tüm isteklere JWT token'ı merkezi bir şekilde otomatik eklemek ve çekilen fotoğrafı multipart form-data formatında sunucuya göndermeyi kolaylaştırmak amacıyla seçilmiştir. Ayrıca kullanıcı deneyimini korumak amacıyla tüm API isteklerinde async/await yapıları kullanılabilir.

expo-secure-store, Cihazın şifrelenmiş depolama alanlarına erişebildiğinden JWT token bilgisini bu alanlarda saklamak ve uygulama kapanıp açılrsa da oturumun güvenli şekilde devam etmesini sağlamak amacıyla seçilmiştir.

Backend katmanı

Framework / Altyapı: .NET 8 (C#), seçimin ana nedeni proje isterlerinde backend tarafında .NET (C#) kullanılmasının istenmesidir. Eşzamanlı olarak gelecek istekleri ve sahadan gönderilecek fotoğrafların yüklenme süreçlerini yüksek performansla yönetebilmesi önemli avantajlarıdır. C# dilinin type-safety yapısı sayesinde kullanıcı ve görev bilgilerindeki bazı hataların daha geliştirme aşamasında fark edilmesi sağlanır. Ayrıca JWT tabanlı kimlik doğrulama, Middleware yapısı ve Dependency Injection desteği sayesinde daha düzenli ve güvenli bir REST API geliştirmek mümkündür.

API Mimarisi: REST, mobil uygulama ile backend servisleri arasındaki iletişimde kullanılmak üzere tercih edilmiştir. Seçimin ana nedeni, REST'in mobil-backend iletişiminde en yaygın, hafif ve ölçeklenebilir standart olmasıdır. HTTP protokolü üzerindeki standart fiilleri (GET, POST, PATCH, DELETE) kullanarak durumsuz (stateless) bir iletişim sağlaması önemli bir avantajıdır. GraphQL veya gRPC gibi alternatiflere kıyasla axios ve .NET Web API ekosistemiyle sıfır yapılandırmayla uyumlu çalışması, JSON formatında veri transferi yapması ve MVP ölçeğindeki saha yönetim uygulamamızın ağ süreçlerini basit ve stabil şekilde yönetmesi tercih edilme sebepleridir.

ORM/Veri Tabanı Erişimi: Entity Framework Core (EF Core), .NET ekosistemiyle tam entegre çalışan standartlaşmış ORM çözümdür. Veritabanı sorgularını SQL yazmak yerine C# ve LINQ ile yazma imkânı sunmasından ötürü seçilmiştir. Ayrıca veritabanı tabloları ve ilişkilerinin doğrudan C# sınıfları üzerinden yönetilmesini sağlar. Böylece ilişkisel veri tabanı işlemleri minimum eforla ve performanslı şekilde yürütülebilir.

Güvenlik/ Kimlik Doğrulama: JWT (JSON Web Token), kullanıcı kimlik doğrulama ve yetkilendirme işlemlerinde kullanılmak üzere tercih edilmiştir. .NET 8 Web API tarafında JWT desteği bulunması ve React Native uygulamasından Axios üzerinden token gönderilebilmesi, bu yapının kullanılmasını kolaylaştırmaktadır.

Kütüphaneler:

BCrypt.Net-Next, kullanıcı şifrelerini veritabanına kaydetmeden önce hash'lemek amacıyla seçilmiştir. Register endpoint'ine düz metin olarak gelen Password, veritabanına yazılmadan önce bu kütüphane ile hash'lenerek PasswordHash alanında saklanır.

Microsoft.AspNetCore.Authentication.JwtBearer, .NET 8 Web API tarafında gelen isteklerdeki JWT token'ların doğrulanması (middleware seviyesinde imza ve geçerlilik kontrolü) için kullanılmıştır.

Npgsql.EntityFrameworkCore.PostgreSQL, EF Core'un PostgreSQL ile haberleşebilmesini sağlayan sağlayıcı (provider) paketidir ve veritabanı bağlantısı bu kütüphane üzerinden kurulmuştur.

Veritabanı katmanı:

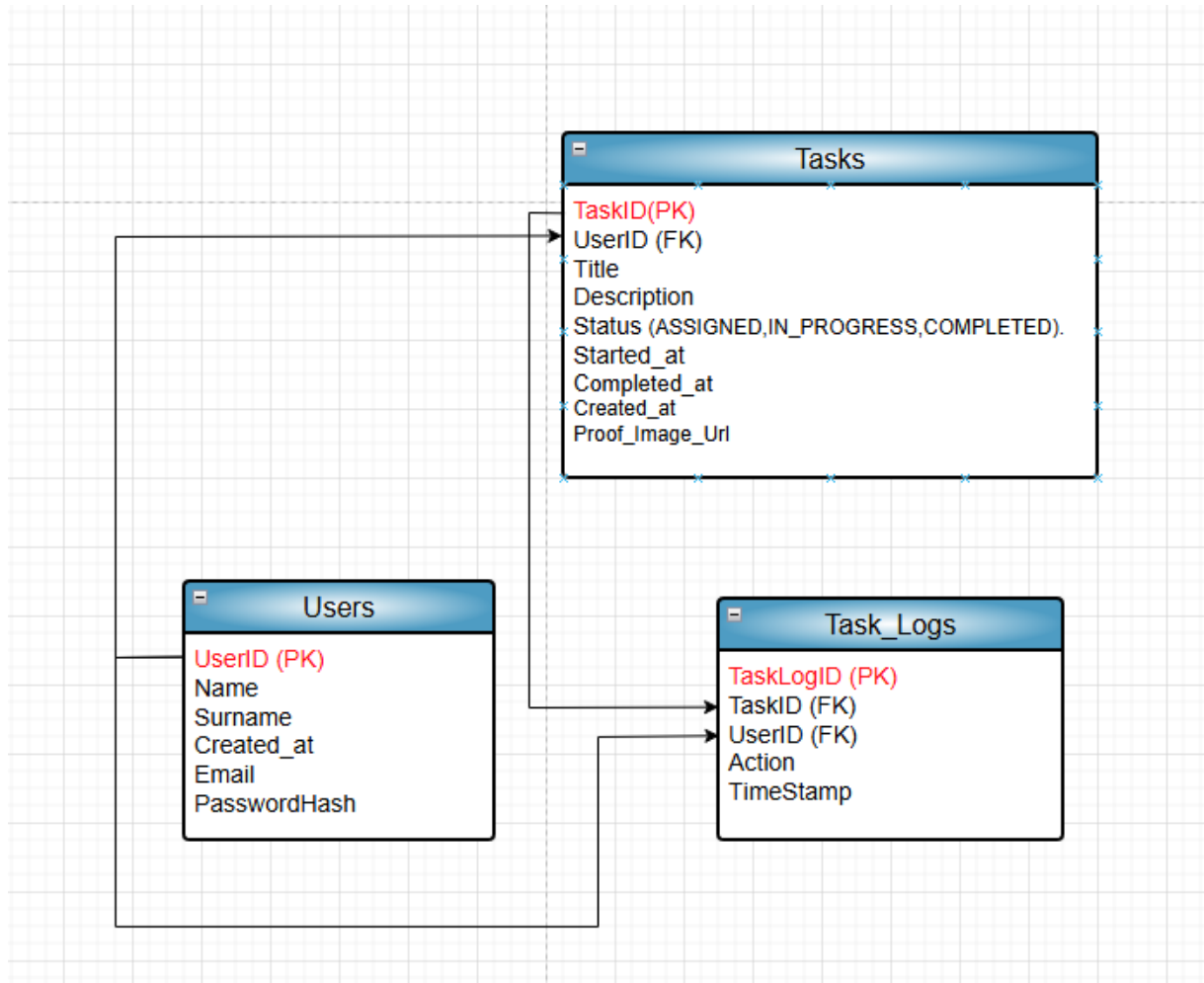
PostgreSQL açık kaynak kodlu, yüksek performanslı olması, daha önceki seçimlerimiz olan .NET 8 ve EF Core ile uyumu (Npgsql sağlayıcısı üzerinden) seçilmesinin sebepleridir. Ayrıca Lisans ücreti olmaması uygulamamızın MVP aşaması için değerlidir.

Medya saklama:

Sunucu dosya sistemi, sahadan çekilen kanıt fotoğraflarının depolanması için tercih edilmiştir. Mobil uygulamadan .NET 8 API'ye gönderilen fotoğraflar, sunucu üzerinde bulunan (wwwroot/uploads/tasks/) klasörde saklanır. Fotoğraflar doğrudan PostgreSQL veritabanına kaydedilmez. Bunun yerine Tasks tablosunda fotoğrafın sunucudaki dosya yolu (/uploads/tasks/task_5_proof.jpg) tutulur. Proje MVP aşamasında olduğu için bu yapının yeterli olacağı düşünülmüş ve şimdilik harici bir bulut depolama servisi kullanılmamıştır.

Veri Akışı ve Veritabanı Tasarımı

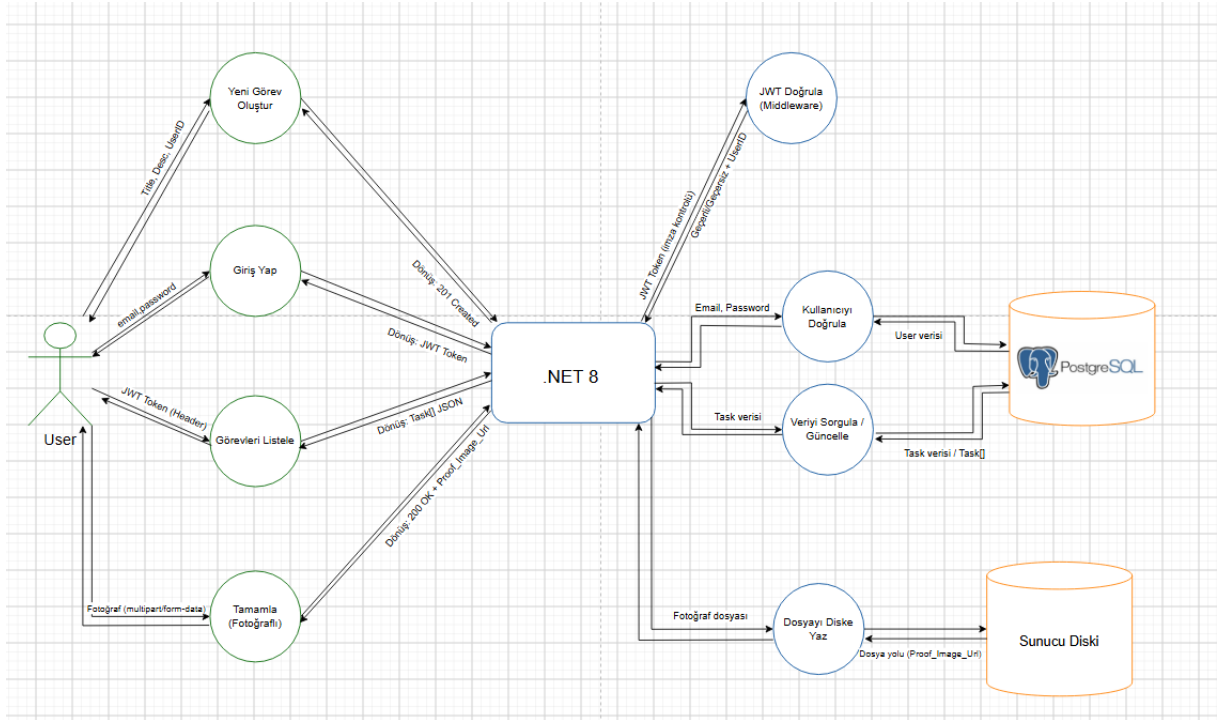
Database schema



Temel API uçları (Endpoints)

Metod	ENDPOINT	AÇIKLAMA	GÖNDERİLEN/ALINAN
POST	/api/v1/auth/register	Yeni kullanıcı kaydı oluşturur	Request: Name, Surname, Email, Password Response: 201 Created
POST	/api/v1/auth/login	Kullanıcı girişi yapar ve yetki anahtarı üretir	Request: Email, Password Response: JWT Token
POST	/api/v1/tasks	Yeni görev oluşturur ve bir personele atar. (MVP olduğu için admin ayrımı yok.)	Request: Title, Description, UserID Response: 201 Created (Status: ASSIGNED)
GET	/api/v1/tasks/my-tasks	Giriş yapan Kullanıcının üzerine atanan görevleri getirir	Header: Bearer <Token> Response: Task[] (JSON Liste)
GET	/api/v1/tasks/{taskId}	Seçilen tek bir görevin detayını getirir	Response: Görev Detayı & Proof_Image_Url
PATCH	/api/v1/tasks/{taskId}/start	Görevi başlatır (IN_PROGRESS) ve zamanı kaydeder	Response: 200 OK (Log kaydı otomatik oluşur)
POST	/api/v1/tasks/{taskId}/complete	Kanıt fotoğrafını alıp sunucuya kaydeder, görev tamamlar (COMPLETED) ve log oluşturur	Request: multipart/form-data (Fotoğraf) Response: 200 OK + Proof_Image_Url
GET	/api/v1/tasks/{taskId}/logs	İlgili göreve ait log kayıtlarını listeler	Response: Task_Logs[] (Zaman sıralı liste)

Örnek Veri Akışı



Veri akışı şemasında 8 farklı endpoint karmaşıklığı önlemek amacıyla 4 temel akışa ayrılmıştır.

Kimlik doğrulama akışını (auth) temsilen Giriş Yap

Veri okuma ve listeleme (read) akışını temsilen Görevleri Listele

Durum güncelleme ve loglama (write) akışını temsilen Yeni Görev Oluştur

dosya yükleme akışını temsilen Tamamla(fotoğraflı) (hem disk hem db ye gittiğinden write akışından ayrı olarak listelenmiştir.)